

Laboratório 5 — Árvore B (ordem 2): função intervalo

1. Identificação

- **Aluno:** Daniel Santana Souza — **Matrícula:** 2310995
- **Disciplina:** INF 1010 — Estruturas de Dados Avançadas (PUC-Rio)
- **Trabalho:** Árvore B de ordem 2 — função de impressão por intervalo

2. Objetivo

O trabalho consiste em representar em C uma árvore B de ordem 2 (na qual cada nó comporta até 4 chaves e 5 ponteiros para subárvores) e implementar uma função capaz de percorrer essa árvore em ordem simétrica imprimindo apenas as chaves que estejam estritamente dentro de um intervalo informado. Ou seja, dados dois limites `lim_inf` e `lim_sup`, a função deve listar, em ordem crescente, todas as chaves x que satisfaçam $\text{lim_inf} < x < \text{lim_sup}$. Além de escrever a função, o objetivo inclui montar uma árvore de exemplo, compilar, gerar o executável e validar o comportamento com três faixas de limites diferentes.

3. Estrutura do programa

O programa foi dividido em módulos para separar a definição/manipulação da estrutura de dados do programa de teste:

`arvoreB.h`

Cabeçalho do módulo da árvore B. Define as constantes `MAX` (4) e `MIN` (2), o tipo `t_no` (o nó da árvore, exatamente como descrito no enunciado, com os campos `ndesc`, `chave[MAX]` e `ramo[MAX+1]`) e declara os protótipos das funções públicas do módulo.

`arvoreB.c`

Implementação do módulo da árvore B. Funções:

- `cria_no(nchaves, chaves, ramos)` — aloca um nó, copia as chaves e os ponteiros de subárvore informados e define `ndesc = nchaves + 1`.
- `cria_folha(nchaves, chaves)` — atalho para criar nós folha (todos os ramos apontam para `NULL`).
- `intervalo(arv, lim_inf, lim_sup)` — **função pedida no enunciado.** Percorre a árvore em ordem simétrica e imprime as chaves no intervalo aberto $(\text{lim_inf}, \text{lim_sup})$.

- `em_ordem(arv)` — percorre e imprime todas as chaves em ordem crescente (usada para conferir a árvore montada).
- `libera_arvore(arv)` — libera recursivamente toda a memória alocada.

`main.c`

Programa principal. Função `constroi_arvore()` monta "à mão" a árvore B de exemplo (3 níveis, balanceada); `testa_intervalo()` executa e imprime um caso de teste; e `main()` exibe a árvore completa e roda os três casos pedidos.

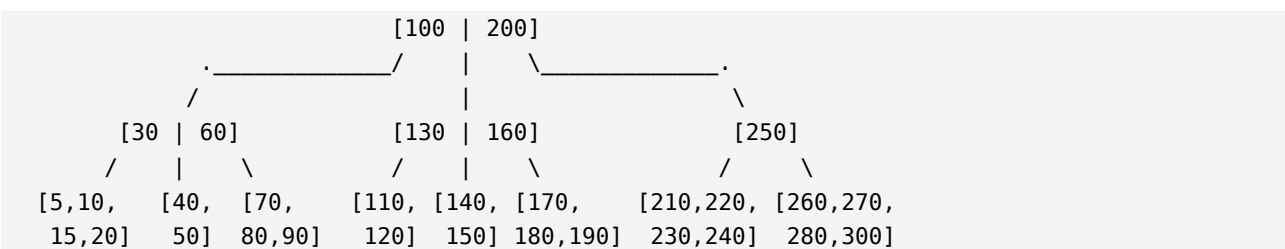
4. Solução

4.1. Representação do nó

Cada nó segue a estrutura do enunciado. A convenção adotada é a de que um nó com k chaves tem $ndesc = k + 1$ ponteiros de ramo. Em um nó folha esses ponteiros apontam todos para NULL, mas $ndesc$ continua valendo $k + 1$, o que torna a travessia uniforme entre nós internos e folhas. As chaves de cada nó ficam sempre em ordem crescente, e `ramo[i]` aponta para a subárvore cujas chaves são menores que `chave[i]` (e maiores que `chave[i-1]`).

4.2. Árvore de exemplo

Como o enunciado pedia a construção manual da árvore, foi montada uma árvore B de ordem 2 válida (todos os nós, exceto a raiz, com pelo menos $MIN = 2$ chaves) e balanceada, com chaves cobrindo a faixa dos limites de teste:



A travessia em ordem simétrica produz todas as chaves em ordem crescente:

```

5 10 15 20 30 40 50 60 70 80 90 100 110 120 130 140 150 160
170 180 190 200 210 220 230 240 250 260 270 280 300
  
```

4.3. A função `intervalo` passo a passo

A ideia é executar uma travessia em ordem simétrica, mas visitando/descendo apenas o que for necessário. Para um nó com $k = ndesc - 1$ chaves, a ordem simétrica visita: `ramo[0]`, `chave[0]`, `ramo[1]`, `chave[1]`, ..., `chave[k-1]`, `ramo[k]`.

O algoritmo percorre as chaves do nó da esquerda para a direita e, para cada chave i :

1. **Desce à subárvore da esquerda** (`ramo[i]`) somente se `chave[i] > lim_inf`. Se `chave[i] <= lim_inf`, todas as chaves dessa subárvore são `<= chave[i]` e, portanto, nenhuma pode estar no intervalo — então a descida é evitada.
2. **Imprime a chave** se `lim_inf < chave[i] < lim_sup`.
3. **Encerra o nó** assim que encontra `chave[i] >= lim_sup`, pois todas as chaves seguintes do nó (e suas subárvores à direita) são ainda maiores e estão fora do intervalo.

Se o laço terminar sem atingir o limite superior, desce-se à subárvore mais à direita (`ramo[k]`). Essas podas evitam visitar ramos inteiros que estão fora da faixa, mantendo a saída sempre em ordem crescente.

4.4. Saída do programa

A execução do programa produz a seguinte saída:

```
Arvore B de ordem 2 (em ordem simetrica):
 5 10 15 20 30 40 50 60 70 80 90 100 110 120 130 140 150 160 170 180 190 200 210 220 230 240
250 260 270 280 300

=== Testes da funcao intervalo() ===

intervalo [5, 300] -> chaves x com 5 < x < 300:
 10 15 20 30 40 50 60 70 80 90 100 110 120 130 140 150 160 170 180 190 200 210 220 230 240
250 260 270 280

intervalo [50, 100] -> chaves x com 50 < x < 100:
 60 70 80 90

intervalo [70, 135] -> chaves x com 70 < x < 135:
 80 90 100 110 120 130
```

Analisando os casos pedidos:

- **a) [5, 300]:** como o intervalo é aberto, 5 e 300 são excluídos; o resto da árvore é listado (de 10 a 280).
- **b) [50, 100]:** apenas 60, 70, 80 e 90 ficam estritamente entre 50 e 100 (50 e 100 ficam de fora).
- **c) [70, 135]:** lista 80, 90, 100, 110, 120 e 130 (70 é excluído por ser o limite inferior).

Todos os resultados conferem com a inspeção manual da árvore.

5. Observações e conclusões

Como compilar e executar

A partir do diretório `lab5`:

```
gcc -Wall arvoreB.c main.c -o arvoreB
./arvoreB
```

O programa não lê arquivos de entrada: a árvore de exemplo é construída diretamente no código (`constroi_arvore`), e os três casos de teste já estão fixados em `main`.

Facilidades e dificuldades

- A maior atenção foi com o **intervalo aberto** (`<` em vez de `<=`): os próprios limites não entram no resultado, o que muda as três saídas.
- Tratar nós internos e folhas de forma uniforme exigiu definir bem a convenção do campo `ndesc` (`k + 1` chaves sempre, com ramos `NULL` nas folhas). Com isso, a recursão de `intervalo` funciona igual para qualquer nó.
- As podas (evitar descer ramos fora da faixa e encerrar o nó ao passar do limite superior) deixam a função eficiente sem comprometer a ordem crescente da saída.

Resultados dos testes

O programa **compila sem warnings** (`-Wall -Wextra`) e **funciona** para os três casos pedidos, produzindo exatamente as chaves esperadas dentro de cada intervalo aberto. Não foram observados problemas de funcionamento; a memória alocada é liberada ao final com `libera_arvore`.